Vanish

Self-destructing secret notes, containerised with Docker and deployed on Amazon EC2

Member A · Member B · Member C

November 2026

Cloud Computing Project, Your College Name

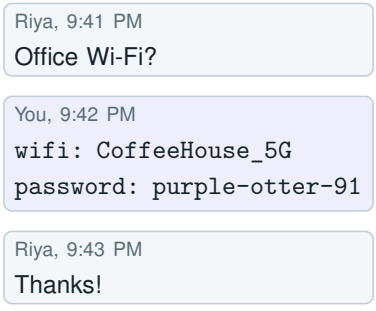
Agenda

1. The problem and our objectives
2. What Vanish does: a short demo
3. Architecture on Amazon EC2
4. Implementation: encryption, backend
5. Docker, Docker Compose and Caddy
6. Building and deploying to EC2
7. Testing, security and cost
8. Lessons learned and future work

The problem: secrets live forever in chat history

- We share passwords, Wi-Fi keys and API keys over WhatsApp, email and Slack.
- Those messages stay in history, cloud backups and screenshots **indefinitely**.
- Existing “secret sharing” sites ask you to trust their server with the plain text.

A secret should be readable once, by the right person, and then disappear.

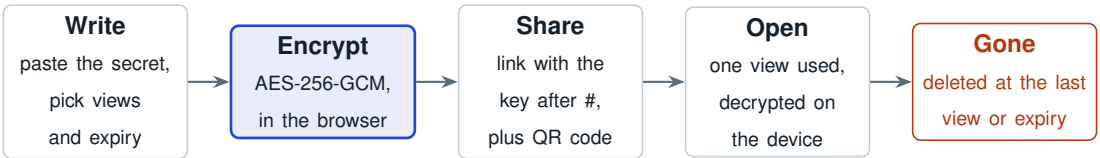


Still there in 2030.

Objectives

1. **Self-destruct**: a note expires after N views or T time, whichever comes first.
2. **Zero knowledge**: the server stores only ciphertext and can never read a note.
3. **Correct under load**: two people can never both get the last view.
4. **Containerised**: the whole app runs from Docker images with one command.
5. **Deployed on a public cloud**: one Amazon EC2 instance, served over HTTPS.

How it works for a user



1 to 10 views

chosen by the sender

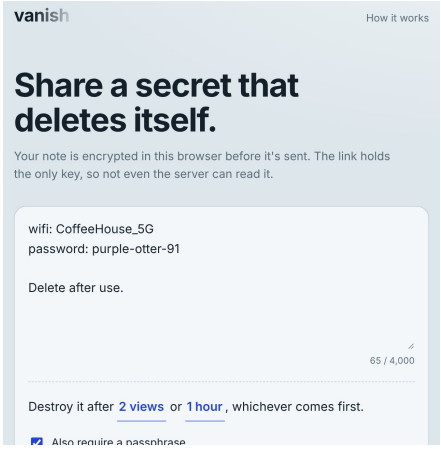
5 min to 7 days

whichever limit comes first

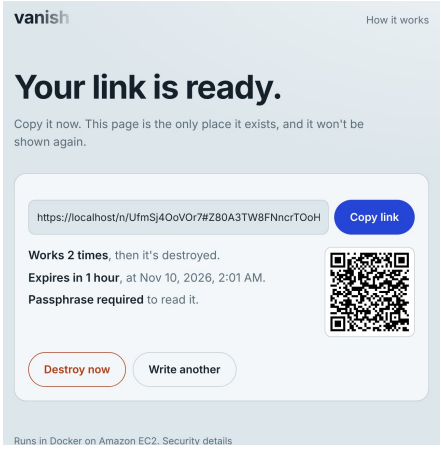
Optional passphrase

a second layer of encryption

Demo: creating a secret

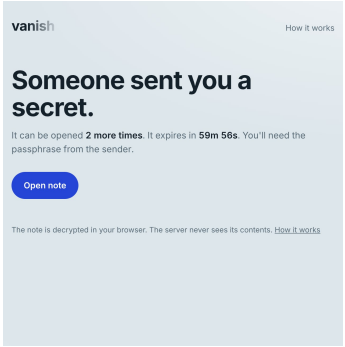


Write the note; 2 views or 1 hour; passphrase on

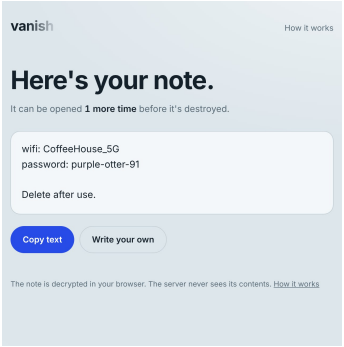


Encrypted in the browser; link and QR code returned

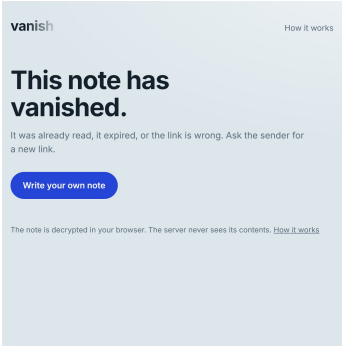
Demo: receiving it, then it vanishes



Countdown; opening the page uses no view

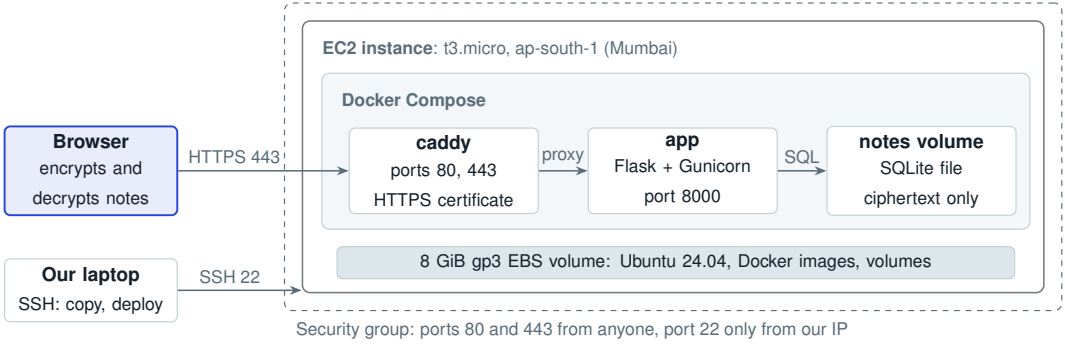


Decrypted on the recipient's device



After the last view: nothing left

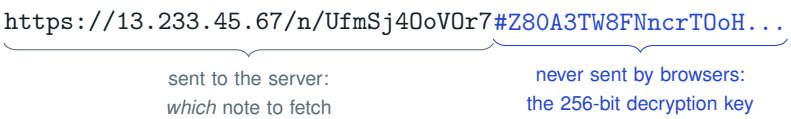
Architecture on Amazon EC2



Cloud services and features used

Service / feature	How Vanish uses it	Setting
EC2 instance	Runs Docker and both containers	t3.micro, 2 vCPU, 1 GiB
AMI	Operating system image	Ubuntu Server 24.04 LTS
User data	Installs Docker and swap on first boot	`deploy/user-data.sh`
Instance metadata	`deploy.sh` looks up the public IP (IMDSv2)	169.254.169.254
Security group	Stateful firewall in front of the instance	22 (our IP), 80, 443
Key pair	Password-less SSH login	RSA, `.pem`
EBS volume	Disk for OS, images and the notes database	8 GiB gp3
Public IPv4	Address users connect to; HTTPS certificate is for it	auto-assigned
EC2 monitoring	CPU, network and status checks	Monitoring tab

Zero-knowledge encryption: the key never reaches the server



- Browser generates a random 256-bit key and encrypts with **AES-256-GCM** (Web Crypto API).
- Only `{ciphertext, iv}` is uploaded.
- Optional passphrase: second layer, key from **PBKDF2-SHA256**, 310,000 rounds.
- GCM is authenticated: tampered data fails to decrypt.

The API: five small endpoints

Method	Path	What it does
`POST`	`/api/notes`	Store `ciphertext`, `iv`, view limit (1–10), expiry (5 min–7 days)
`GET`	`/api/notes/{id}`	Views left and expiry; **does not use a view** (safe for link previews)
`POST`	`/api/notes/{id}/open`	Atomically use one view and return the ciphertext; `410 Gone` afterwards
`DELETE`	`/api/notes/{id}`	Destroy early with the secret delete token (stored as a SHA-256 hash)
`GET`	`/api/health`	Liveness check used by the Docker health check

A background thread deletes expired notes every 60 seconds; every read also checks the expiry time.

Atomic view counting: exactly one winner

```
UPDATE notes
   SET view_count = view_count + 1
 WHERE id = ?
   AND view_count < max_views
   AND expires_at > ?
RETURNING ciphertext, iv, max_views,
          view_count, expires_at;
```

From `open_note` in `app/main.py`. If no row comes back, the API answers `410 Gone`.

Why one statement?

Check and update happen together, so two people opening the *last* view at the same moment can't both succeed.

Tested

8 simultaneous requests on a 1-view note: **1** gets the note, **7** get 410.

Containerising the app: the Dockerfile

```
FROM python:3.13-slim
ENV DB_PATH=/data/vanish.db
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir \
        -r requirements.txt
COPY app/ .
RUN useradd --system --uid 10001 vanish \
 && mkdir -p /data \
 && chown vanish:vanish /data
USER vanish
EXPOSE 8000
HEALTHCHECK CMD python -c "..."  # GET /api/health
CMD ["gunicorn", "--bind", "0.0.0.0:8000",
     "--workers", "1", "--threads", "8",
     "main:app"]
```

- **Slim base image**: small download, fewer packages to patch.
- **Dependencies first**: code changes don't re-run `pip`.
- **Non-root user**: a compromised app can't take over the container.
- **Health check**: Docker flags the container if the API stops answering.
- **Gunicorn**: production web server, 8 threads.

Two containers: Docker Compose and Caddy

compose.yaml (abridged)

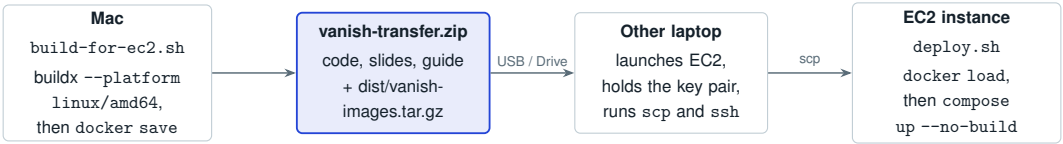
```yaml
services:
  app:
    image: vanish-app:1.0
    build: .
    volumes: ["notes:/data"]
    ports: ["127.0.0.1:8000:8000"]   # private
  caddy:
    image: caddy:2.11
    ports: ["80:80", "443:443"]       # public
    environment:
      SITE_ADDRESS: ${SITE_ADDRESS}
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
```

Caddyfile

```
{
  default_sni {$SITE_ADDRESS}
}
{$SITE_ADDRESS} {
  tls {
    issuer acme {
      profile shortlived
    }
  }
  encode zstd gzip
  reverse_proxy app:8000
}
```

Free Let's Encrypt certificate for the public IP, renewed automatically.

Build once, ship the image to EC2



Why `linux/amd64`?

Apple Silicon Macs build `arm64` images by default, but a t3.micro has an Intel/AMD processor. A wrong image fails with *exec format error*, so `deploy.sh` checks the architecture before starting.

Deploying on EC2 in four steps

1. **Launch** a t3.micro with Ubuntu 24.04, key pair `vanish-key`, HTTP + HTTPS open, SSH from our IP, and `user-data.sh` pasted in.
2. **Copy** the zip with `scp`.
3. **Unzip** and run `deploy.sh`.
4. **Open** `https://<public-ip>`.

```
# from the laptop
scp -i vanish-key.pem vanish-transfer.zip \
    ubuntu@13.233.45.67:~
ssh -i vanish-key.pem ubuntu@13.233.45.67

# on the instance
unzip vanish-transfer.zip && cd vanish
bash deploy/deploy.sh
docker compose ps
docker compose logs caddy   # certificate
```

`deploy.sh` asks the instance metadata service for the public IP, writes it to `.env`, loads the images and starts both containers.

Security measures

Data

- Server stores ciphertext, timestamps and counters only
- Key in the URL fragment; removed from the address bar after opening
- Delete tokens stored as SHA-256 hashes
- Notes up to 4,000 characters; 20 new notes per minute per IP

Infrastructure

- Security group: only ports 22 (our IP), 80 and 443
- App port bound to localhost; only Caddy is public
- Container runs as a non-root user
- HTTPS everywhere, HSTS, strict Content-Security-Policy, no referrer

Testing

Automated tests (pytest)

16 / 16

passing

- View counting and burn-on-last-view
- 8 concurrent opens: exactly 1 winner
- Expiry and cleanup, delete tokens
- Input validation, rate limit, pages

Live checks on the EC2 deployment

1. HTTPS loads; `http://` redirects to `https://`
2. Create a 2-view note; open it twice; third open shows “vanished”
3. Wrong passphrase rejected; right one decrypts
4. *Destroy now* works; missing key detected
5. Port 8000 unreachable from the internet
6. Notes survive `docker compose restart`
7. Database holds ciphertext only

Cost

Item (ap-south-1, 24/7)	USD / month
t3.micro instance ($0.0112 / hour)	$\approx$ 8.20
Public IPv4 address	$\approx$ 3.65
8 GiB gp3 EBS volume	$\approx$ 0.75
Total	$\approx$ **12.60**

Paid by the Free plan

New AWS accounts get **$100 credit**; t3.micro is Free Tier eligible. Our out-of-pocket cost: **INR 0**.

Stopping the instance when idle leaves only the disk charge.

Challenges and lessons learned

- **CPU architecture matters.** Images built on an Apple Silicon Mac are `arm64`; EC2 t3 is `amd64`. Fixed with `buildx --platform linux/amd64` and a check in `deploy.sh`.
- **Browser crypto needs HTTPS.** Web Crypto is disabled on plain `http://`, so Caddy with a Let's Encrypt IP certificate is part of the design.
- **Public IPs change.** Stopping and starting the instance gives a new IP; re-running `deploy.sh` picks it up automatically.
- **1 GiB of RAM is tight.** User data adds 1 GiB of swap so builds on the instance never run out of memory.

Limitations and future work

Limitations

- Anyone holding the full link can read the note; use a passphrase for sensitive data
- One instance: no redundancy if it stops
- SQLite on one disk: fine for this scale, not for many servers

Future work

- Custom domain and an Elastic IP for a stable address
- Managed database (Amazon RDS) and two instances behind a load balancer
- Image registry (Amazon ECR) and a CI pipeline for builds
- File attachments, encrypted the same way

Summary

- Secrets that **self-destruct** after N views or T time
- **Zero-knowledge**: encrypted in the browser, ciphertext on the server
- **Atomic** view counting, verified under concurrency
- Packaged as **Docker images**, run with Docker Compose
- Deployed on **Amazon EC2** over HTTPS, for INR 0

Thank you

Questions?

Member A
Member B
Member C